ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Informática Médica - INFME - PL3

Node.js| Luis Conceição, Diogo Martinho, Vítor Crista

P.PORTO

PL 02 - Sumário

- Introdução ao JavaScript e o seu papel no desenvolvimento web (HTML + CSS + JS).
- Tipos de dados e variáveis em JavaScript.
- Estruturas de dados: objetos e arrays.
- Estruturas de controlo: condicionais (if/else) e ciclos (for, while).
- Funções em JavaScript e integração com HTML.

O que é?

- Ambiente de execução JavaScript no lado do servidor
- Baseado no motor V8 do Google Chrome
- Arquitetura event-driven e non-blocking I/O
- Permite criar APIs, servidores web e aplicações em tempo real
- Grande ecossistema de pacotes via npm (Node Package Manager)
- Muito utilizado em aplicações escaláveis e de alta performance

Grandes empresas que utilizam Node.js

Node.js – Arquitetura event-driven e non-blocking I/O

```
const fs = require('fs');

console.log("Start");

fs.readFile('file.txt', 'utf8', (err, data) => {
    console.log("File loaded");
});

console.log("End");
```

```
Start
End
File loaded
```

- **Non-Blocking I/O**
  - Node.js não espera pela leitura do ficheiro
  - Continua a executar o resto do código
- **Event-Driven**
  - Quando a leitura termina ocorre um evento
  - O callback é executado

ISEP

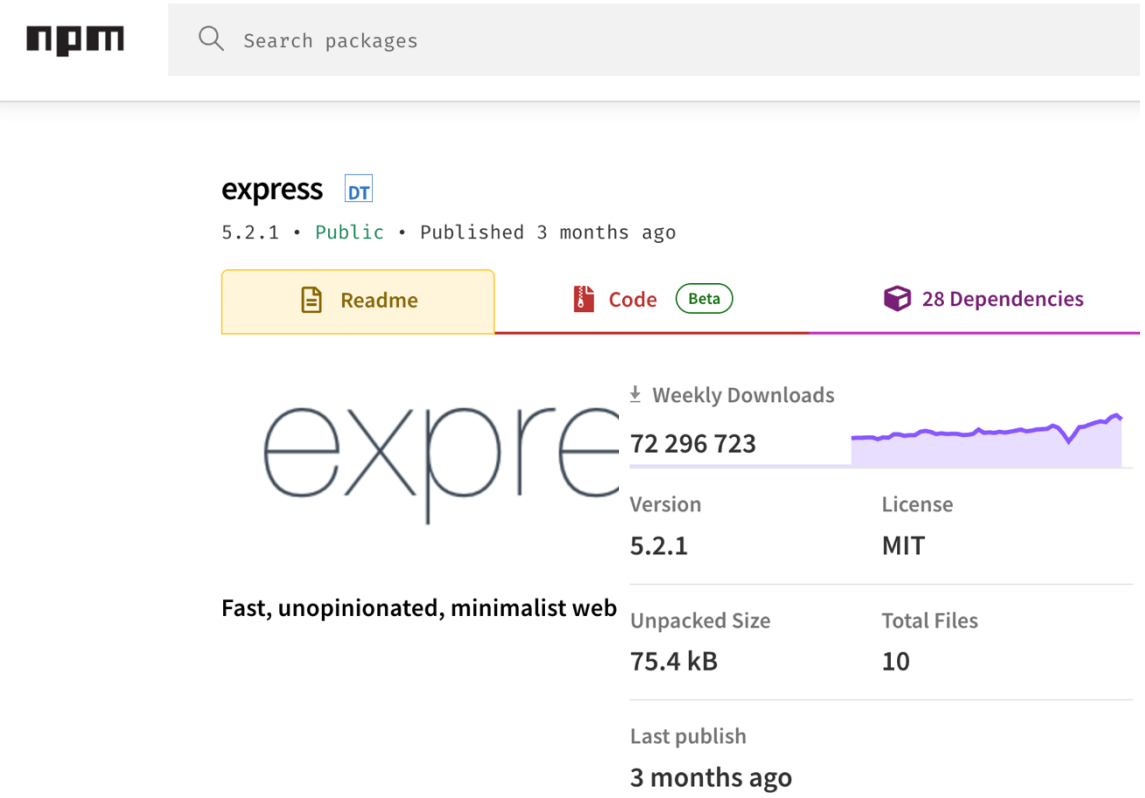
P.PORTO

O que é possível fazer com Node.js?

- O Node.js pode gerar conteúdo dinâmico para páginas.
- O Node.js pode criar, abrir, ler, escrever, apagar e fechar ficheiros no servidor.
- O Node.js pode recolher dados de formulários.
- O Node.js pode adicionar, apagar e modificar dados na base de dados.

Porquê utilizar Node.js?

- Operações de I/O não bloqueantes
- Muitas bibliotecas (módulos)
- Comunidade grande e ativa (listas de e-mail, Twitter, GitHub)
- Suportado pelos principais sistemas operativos
- 2 aplicações (*Frontend* e *Backend*), uma única linguagem



ISEP

P.PORTO

Eventos

- *callback function* (o listener)

```
var listener1 = function() {
    console.log('Execução da função listener 1');
}
```

- Regista o *callback* no emissor de eventos.

```
on('evento1', listener1)
// or
addListener('evento1', listner1)
```

- Ao emitir o evento, a função de *callback* é executada.

```
eventEmitter.emit('evento1');
```

ISEP

P.PORTO

Eventos (Exemplo)

exemplo.js

```
var eventos = require('events');

var eventEmitter = new eventos.EventEmitter();

var listener1 = function() {
   console.log('Executar a função listener');
}
eventEmitter.on('event1', listener1);
eventEmitter.emit('event1');

console.log('Fim do programa');
```

```
C:\> node exemplo.js
Executar a função listener
Fim do programa
C:\>
```

ISEP

P.PORTO

Eventos (Exemplo 2)

exemplo.js

```
var msg;

loadRecords = function (callback) {
  setTimeout(function() {
    //Simula a leitura de registos da base de dados
    msg = ”A minha Tarefa";
    callback();
  }, 3000)
}

printMsg = function () {
  console.log(msg);
}

loadRecords(printMsg);
console.log("Ola");
```

```
C:\> node exemplo.js
Ola
A minha Tarefa
C:\>
```

ISEP

P.PORTO

Eventos (Exemplo 4)

```
const fs = require('fs');
const data = fs.readFileSync(‘/data.json’);   // blocks here until file is read
console.log(data);
moreWork();                                  // will run after console.log
```

```
const fs = require('fs');
fs.readFile(‘/data.json', (err, data) => {
        if (err) throw err;
        console.log(data);
});
moreWork();                                   // will run before console.log
```

ISEP

P.PORTO

Ambiente de Desenvolvimento (Introdução)

NPM - Node Package Manager

- Package manager para a linguagem de programação JavaScript.
- Para aceder é através do cliente.

```
C:\ProjectFolder\>npm install express
```

- Os packages disponíveis podem ser explorados e pesquisados através do website do npm: http://www.npmjs.com.

Alguns packages populares

- **Express.js**
  - Framework minimalista e muito utilizado para criar servidores web e APIs REST em Node.js.
- **Axios**
  - Biblioteca para realizar pedidos HTTP a APIs externas de forma simples e baseada em promises.
- **Mongoose**
  - Biblioteca que facilita a interação com bases de dados MongoDB, oferecendo um modelo orientado a objetos.
- **Socket.IO**
  - Permite comunicação em tempo real entre cliente e servidor através de WebSockets.
- **Passport.js**
  - Middleware para implementar autenticação em aplicações Node.js (login, OAuth, JWT, etc.).
- **Dotenv**
  - Permite carregar variáveis de ambiente a partir de ficheiros .env, facilitando a gestão de configurações.
- **Cors**
  - Middleware usado para configurar Cross-Origin Resource Sharing, permitindo que diferentes domínios acedam à API.

Node.js - Instalação

```
> node_modules
JS index.js
{} package-lock.json
{} package.json
```

- Download Node.js em http://nodejs.org

- Instalar Node.js

```
C:\> node -v
(mostra aqui a versão instalada, e.g. v25.2.1)
```

- Instalar a Framework Express.js

```
C:\> npm install express
```

ISEP

P.PORTO

Ficheiro package.json

- Localizado no diretório raiz do projeto Node.js
- Contém metadados relevantes sobre o projeto:
  - *name*
  - *version*
  - *description*
  - *keywords*
  - *dependencies*
  - *license*
  - *main*
  - *author*
  - …

package.json

```
{
        "name": "node-api",
        "main": "index.js",
        "dependencies": {
                "body-parser": "~1.0.1",
                "express": "~4.0.0",
                "mongoose": "*",
                "mongoose-id-validator": "^0.4.3"
        }
}
```

Aproximadamente equivalente à versão

Corresponde a qualquer versão

Compatível com a versão

ISEP

P.PORTO

Criar um ficheiro package.json genérico

- Executar o comando:
- The *flag* **--yes** ou **-y** indica ao comando para extrair informações do diretório atual.

name	the current directory name
version	always **1.0.0**
description	info from the README, or an empty string ""
main	always **index.js**
scripts	by default, creates an empty test script
keywords	empty
author	empty
license	ISC license
bugs	information from the current directory, if present
homepage	information from the current directory, if present

ISEP

P.PORTO

Node Modules

- Os módulos instalados são carregados pelo código usando a função:

```
require(<Module_Name>)
```

- A funcionalidade básica de require é ler um ficheiro JavaScript, executá-lo e devolver o objeto exportado.
- Exemplo:

example.js

```
var uc = require('upper-case');
console.log(uc("hello world!"));
```

```
C:\> node example.js
HELLO WORLD!
C:\>
```

ISEP

P.PORTO

Primeiro projeto em Node.js

- O módulo HTTP integrado
  - Permite ao Node.js transferir dados através do protocolo HTTP.

```
var http = require('http');
```

- Node.js como servidor web
  - O módulo HTTP pode criar um servidor HTTP que escuta portas do servidor e envia uma resposta de volta ao cliente (método createServer()).

```
var http = require('http');

//create a server object
http.createServer(function (req, res) {
  res.write('Hello World!’);   //write a response to the client
  res.end();                   //end the response
}).listen(8080);          //the server object listens on port 8080
```

ISEP

P.PORTO

ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Informática Médica - INFME

Node.js| Luis Conceição, Diogo Martinho, Vítor Crista

P.PORTO